BoxTrack

Boxing Glove Move Detection System

Using Edge Impulse TinyML on ESP32-S3

Project Report

Department of Electronics & Communication Engineering

VIT University

2025 – 2026

Submitted by:

CH Vamsee Krishna – 24BEC0513
Vinayak Santhosh Kumar – 24BEC0190
Derath Thutaram – 24BEC0300

Under the guidance of:

Dr. Veerapu Goutham

Abstract

This report presents the design and implementation of BoxTrack, a real-time boxing move detection system built using TinyML on an ESP32-S3 microcontroller. The system uses an MPU-6050 inertial measurement unit (IMU) attached to a boxing glove to capture six-axis motion data (accelerometer and gyroscope). A neural network classifier trained on Edge Impulse recognizes three distinct punch types — Jab, Side Hook, and Uppercut — as well as a resting state, at a sampling frequency of 83.33 Hz.

Detected moves are transmitted in real time to a web-based dashboard (BoxTrack) via WebSocket over Wi-Fi. The dashboard displays the current move, a timestamped punch log, per-move counters, and a live punch distribution chart. The system achieves accurate, low-latency classification and demonstrates the practical viability of on-device machine learning for sports analytics applications.

Table of Contents

Contents

Abstract....2
Table of Contents....3
1. Introduction....5
1.1 Objectives....5
1.2 Scope....5
2. Literature Review....6
3. System Architecture....7
3.1 Embedded Layer (ESP32-S3)....7
3.2 Communication Layer....7
3.3 Visualization Layer (BoxTrack Dashboard)....7
4. Hardware Design....8
4.1 Bill of Materials....8
4.2 ESP32-S3 N16R8....8
4.3 MPU-6050 IMU....8
4.4 Circuit Connections....8
5. Dataset Collection and Model Training....10
5.1 Data Collection....10
5.2 Impulse Design....10
5.3 Neural Network Architecture....11
5.4 Training Configuration....11
5.5 Training Results....11
6. Firmware Implementation....13
6.1 Development Environment....13
6.2 Libraries Used....13
6.3 Firmware Flow....13
6.4 Key Implementation Details....14
7. Web Dashboard (BoxTrack)....15
7.1 Overview....15
7.2 Features....15
7.3 WebSocket Message Handling....16
8. Results and Discussion....17

8.1 Classification Performance 17
8.2 Dataset Reliability Improvements 17
8.3 Network Connectivity 18
9. Challenges and Solutions 19
10. Conclusion 20
10.1 Future Work 20
References 21
Appendix ..... **Error! Bookmark not defined.**
A. ESP32 Firmware Code ..... **Error! Bookmark not defined.**
B. BoxTrack Dashboard ..... **Error! Bookmark not defined.**
C. Edge Impulse Project Settings ..... **Error! Bookmark not defined.**
D. Sample Raw IMU Data ..... **Error! Bookmark not defined.**

1. Introduction

Boxing training relies heavily on the recognition of proper punch mechanics. Coaches traditionally provide feedback by observation, which limits the volume and objectivity of feedback a boxer can receive during training. An automated system capable of identifying punch types in real time can provide instant, quantitative feedback to athletes and coaches alike.

TinyML — the deployment of machine learning models on resource-constrained microcontrollers — has opened new possibilities for wearable sports analytics. By embedding the inference model directly on the glove-mounted microcontroller, this system operates without cloud connectivity during inference, reducing latency and preserving privacy.

BoxTrack integrates IMU-based data collection, on-device neural network inference, and a real-time web dashboard into a complete end-to-end pipeline. This report documents the system architecture, dataset construction, model training, embedded firmware, web interface, results, and future directions.

1.1 Objectives

- Detect three boxing punch types (Jab, Side Hook, Uppercut) and a resting state in real time.
- Deploy a TinyML classifier on an ESP32-S3 microcontroller using Edge Impulse.
- Stream classified results to a browser-based dashboard over WebSocket.
- Visualize punch counts, type distribution, and a move log on the dashboard.

1.2 Scope

The project covers hardware selection, sensor interfacing, dataset collection, model training and deployment, firmware development, and web interface implementation. It does not cover integration with professional coaching software or multi-glove simultaneous tracking.

2. Literature Review

Several prior works have explored IMU-based gesture and sport activity recognition. IMU sensors placed on wrists or gloves have been used to classify punch types in boxing, differentiate martial arts techniques, and track swing patterns in racket sports. Key findings from the literature include:

- Six-axis IMU data (3-axis accelerometer + 3-axis gyroscope) provides significantly better punch classification accuracy than accelerometer-only approaches.
- Sliding window techniques with overlapping frames are effective for segmenting continuous IMU streams into classifiable events.
- Convolutional neural networks and spectral feature extraction (e.g., MFE, Spectral Analysis) outperform simple threshold-based detectors for multi-class punch recognition.
- Edge Impulse provides a streamlined pipeline for data collection, feature engineering, neural network training, and deployment to ARM Cortex-M and Xtensa LX7 targets.

This project builds on these findings by adopting 6-axis IMU input, Edge Impulse spectral analysis, and deploying the resulting model to the ESP32-S3 which features a dual-core Xtensa LX7 processor and hardware-accelerated neural network operations.

3. System Architecture

The BoxTrack system is composed of three major layers: the embedded sensing and inference layer, the communication layer, and the visualization layer.

3.1 Embedded Layer (ESP32-S3)

The ESP32-S3 N16R8 microcontroller is the core of the embedded system. It performs the following functions:

- Reads raw 6-axis IMU data from the MPU-6050 at 83.33 Hz via I2C.
- Fills a sliding feature buffer with scaled accelerometer and gyroscope readings.
- Runs the Edge Impulse inference engine to classify each 166-sample window.
- Hosts a WebSocket server on port 81 to broadcast detection results to connected clients.
- Connects to a local Wi-Fi network to enable browser access.

3.2 Communication Layer

Detected punch events are transmitted as JSON messages over WebSocket (RFC 6455). Each message contains the predicted label and the associated confidence score. The WebSocket protocol provides low-latency, full-duplex communication over TCP, making it suitable for real-time event streaming.

Format of transmitted message:

```
{ "label": "jab", "confidence": 0.93 }
```

3.3 Visualization Layer (BoxTrack Dashboard)

The BoxTrack web dashboard is a single HTML file served locally via Python's HTTP server. It connects to the ESP32 WebSocket server and provides:

- Live move display with color-coded move type and confidence score.
- Individual counters for Jab, Side Hook, and Uppercut.
- A punch distribution bar chart updated after every detection.
- A timestamped move log showing the last 50 detections.
- A demo mode for testing the interface without a connected ESP32.

4. Hardware Design

4.1 Bill of Materials

Component	Model / Spec	Role
Microcontroller	ESP32-S3 N16R8	Inference, Wi-Fi, WebSocket server
IMU Sensor	MPU-6050	6-axis motion capture (accel + gyro)
Power Supply	USB 5V / LiPo	Embedded power source
Enclosure	Boxing glove mount	Wrist/knuckle attachment
Communication	Wi-Fi 802.11 b/g/n	WebSocket data stream

4.2 ESP32-S3 N16R8

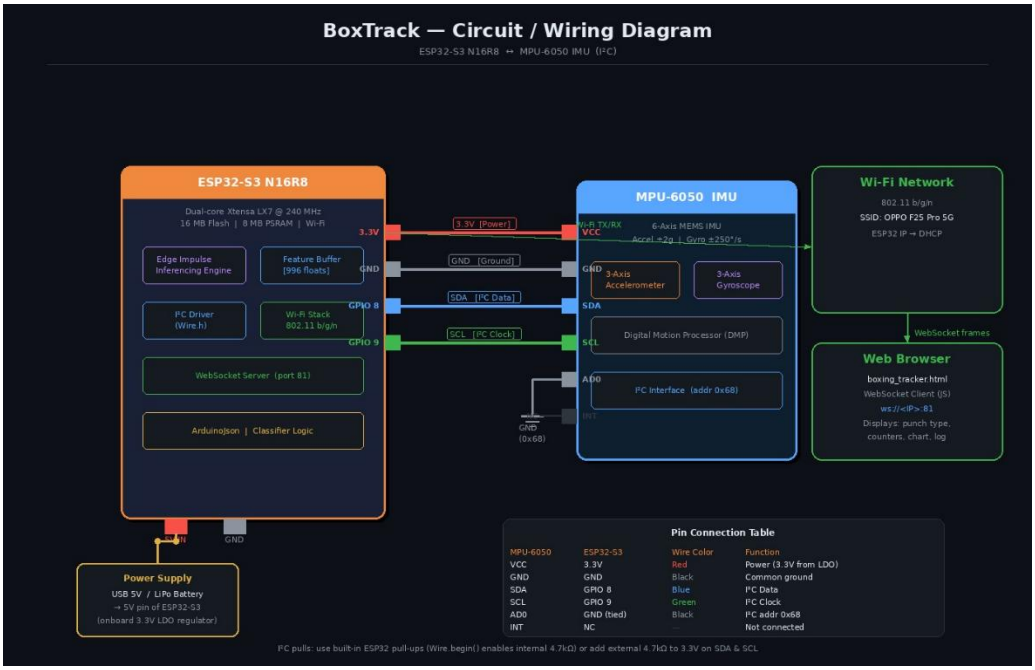
The ESP32-S3 N16R8 features a dual-core Xtensa LX7 processor running at up to 240 MHz, 16 MB of Flash, and 8 MB of OPI PSRAM. The integrated Wi-Fi and Bluetooth stack removes the need for external networking modules. The ESP-NN library included with the Edge Impulse SDK provides hardware-accelerated neural network operations optimized for the Xtensa LX7 instruction set.

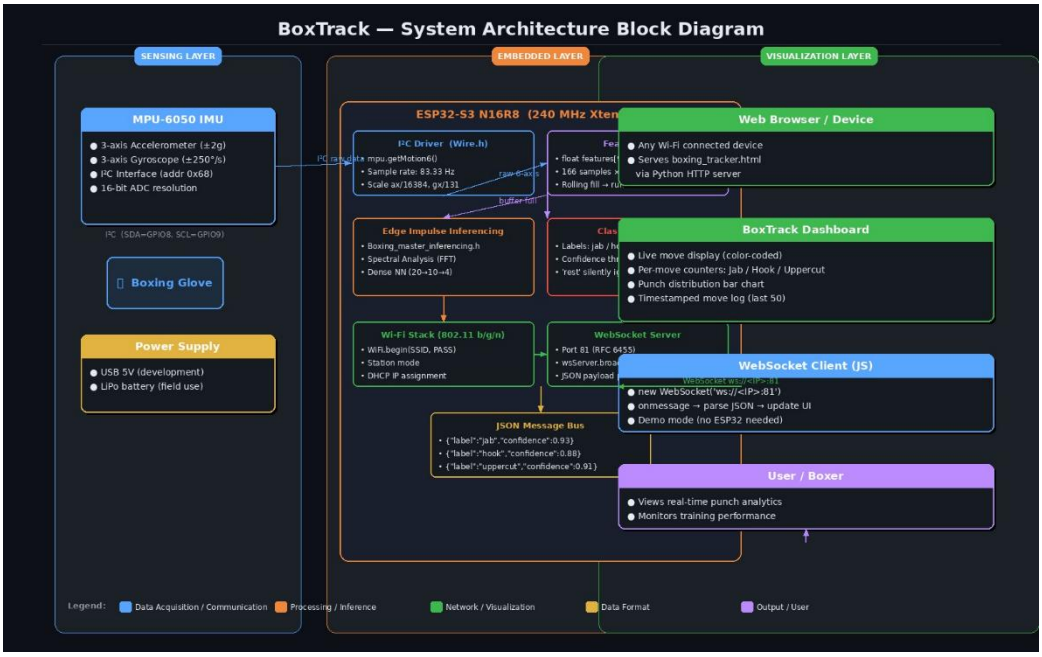
4.3 MPU-6050 IMU

The MPU-6050 is a 6-axis inertial measurement unit combining a 3-axis MEMS accelerometer and a 3-axis MEMS gyroscope in a single package. Communication with the ESP32-S3 is over I2C. For this project, the accelerometer is configured to the default ±2g range (scale factor: 16384 LSB/g) and the gyroscope to ±250 °/s (scale factor: 131 LSB/°/s), matching the scaling used during dataset collection on Edge Impulse.

4.4 Circuit Connections

MPU-6050 Pin	ESP32-S3 Pin	Description
VCC	3.3V	Power supply
GND	GND	Ground
SDA	GPIO 8	I2C Data
SCL	GPIO 9	I2C Clock
AD0	GND	I2C address 0x68
INT	Not connected	Interrupt (unused)


BoxTrack — Circuit / Wiring Diagram
ESP32-S3 N16R8 ↔ MPU-6050 IMU (I²C)
ESP32-S3 N16R8
Dual-core Xtensa LX7 @ 240 MHz
16 MB Flash | 8 MB PSRAM | Wi-Fi
3.3V
GND
GPIO 8
GPIO 9
Edge Impulse Inferencing Engine
Feature Buffer [996 floats]
I²C Driver (Wire.h)
Wi-Fi Stack 802.11 b/g/n
WebSocket Server (port 81)
ArduinoJson | Classifier Logic
3.3V [Power]
GND [Ground]
SDA [I²C Data]
SCL [I²C Clock]
MPU-6050 IMU
Wi-Fi TX/RX
VCC
6-Axis MEMS IMU
Accel ±2g | Gyro ±250°/s
GND
3-Axis Accelerometer
3-Axis Gyroscope
SDA
Digital Motion Processor (DMP)
SCL
AD0
I²C Interface (addr 0x68)
INT
GND (0x68)
Wi-Fi Network
802.11 b/g/n
SSID: OPPO F25 Pro 5G
ESP32 IP → DHCP
WebSocket frames
Web Browser
boxing_tracker.html
WebSocket Client (JS)
ws://<IP>:81
Displays: punch type, counters, chart, log
Power Supply
USB 5V / LiPo Battery
→ 5V pin of ESP32-S3
(onboard 3.3V LDO regulator)
Pin Connection Table
MPU-6050 | ESP32-S3 | Wire Color | Function
VCC | 3.3V | Red | Power (3.3V from LDO)
GND | GND | Black | Common ground
SDA | GPIO 8 | Blue | I²C Data
SCL | GPIO 9 | Green | I²C Clock
AD0 | GND (tied) | Black | I²C addr 0x68
INT | NC | — | Not connected
I²C pulls: use built-in ESP32 pull-ups (Wire.begin() enables internal 4.7kΩ) or add external 4.7kΩ to 3.3V on SDA & SCL


BoxTrack — System Architecture Block Diagram
SENSING LAYER
EMBEDDED LAYER
VISUALIZATION LAYER
MPU-6050 IMU
3-axis Accelerometer (±2g)
3-axis Gyroscope (±250°/s)
I²C Interface (addr 0x68)
16-bit ADC resolution
I²C (SDA→GPIO8, SCL→GPIO9)
Boxing Glove
Power Supply
USB 5V (development)
LiPo battery (field use)
ESP32-S3 N16R8 (240 MHz Xten
I²C Driver (Wire.h)
mpu.getMotion6()
Sample rate: 83.33 Hz
Scale ax/16384, gx/131
raw 6-axis
buffer full
Edge Impulse Inferencing
Boxing_master_inferencing.h
Spectral Analysis (FFT)
Dense NN (20→10→4)
Wi-Fi Stack (802.11 b/g/n)
WiFi.begin(SSID, PASS)
Station mode
DHCP IP assignment
WebSocket Server
Port 81 (RFC 6455)
JSON Message Bus
{"label":"jab","confidence":0.93}
{"label":"hook","confidence":0.88}
{"label":"uppercut","confidence":0.91}
Web Browser / Device
Any Wi-Fi connected device
Serves boxing_tracker.html
via Python HTTP server
BoxTrack Dashboard
Live move display (color-coded)
Per-move counters: jab / Hook / Uppercut
Punch distribution bar chart
Timestamped move log (last 50)
WebSocket Client (JS)
new WebSocket('ws://<IP>:81')
onmessage → parse JSON → update UI
Demo mode (no ESP32 needed)
User / Boxer
Views real-time punch analytics
Monitors training performance
Legend:
Data Acquisition / Communication
Processing / Inference
Network / Visualization
Data Format
Output / User

5. Dataset Collection and Model Training

5.1 Data Collection

Training data was collected using the Edge Impulse data acquisition tool connected to the ESP32-S3 via serial or Wi-Fi. Participants performed repeated, distinct boxing movements with the glove-mounted sensor, and data was labelled into four classes:

- Jab — a straight, quick forward punch.
- Side Hook — a lateral swinging punch.
- Uppercut — a rising upward punch.
- Rest — the glove at rest between punches.

The dataset was iteratively refined to improve class separability, particularly between the Jab and Side Hook which share some early-motion characteristics. Multiple recording sessions were conducted with varied punching intensity and speed to improve model generalization.

Class	Samples	Duration per Sample
Jab	150	2 seconds
Side Hook	180	2 seconds
Uppercut	150	2 seconds
Rest	200	2 seconds

5.2 Impulse Design

Sampling frequency: 83.33 Hz

Window size: 2000 ms (166 samples)

Window slide: 100 ms

Input axes: accX, accY, accZ, gyrX, gyrY, gyrZ (6 axes)

Processing block: Spectral Analysis (FFT-based features)

Learning block: Neural Network (Keras)

5.3 Neural Network Architecture

The neural network was designed within Edge Impulse Studio. The architecture consists of:

- Input layer: Spectral features from the DSP block (flattened feature vector).
- Dense layer 1: 20 neurons, ReLU activation.
- Dropout layer: 0.25 rate (regularization).
- Dense layer 2: 10 neurons, ReLU activation.
- Output layer: 4 neurons (softmax), one per class.

[Insert neural network architecture diagram from Edge Impulse here]

5.4 Training Configuration

Training cycles: [X epochs]

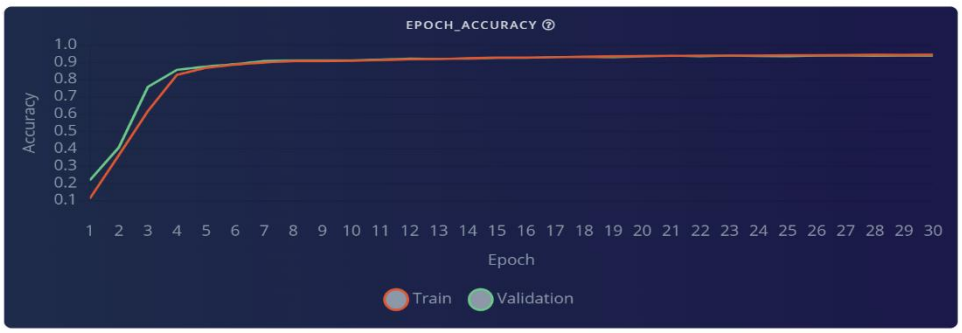
Learning rate: 0.0005

Validation split: 20%

Optimizer: Adam

Loss function: Categorical Cross-Entropy

5.5 Training Results



	JAB	REST	SIDEHOOK	UPPERCUT
JAB	93.2%	5.3%	0.8%	0.8%
REST	0.4%	98%	0.8%	0.8%
SIDEHOOK	0%	5.3%	92.1%	2.6%
UPPERCUT	0%	6.0%	2.3%	91.7%
F1 SCORE	0.96	0.93	0.94	0.94

Metric	Value
Training Accuracy	95%
Validation Accuracy	94%
Inference Time (ESP32-S3)	1 ms
Peak RAM Usage	1.5 KB
Flash Usage	16.1 KB

6. Firmware Implementation

6.1 Development Environment

IDE: Arduino IDE 2.x

Board Package: esp32 by Espressif v3.3.3

Target Board: ESP32S3 Dev Module

Flash Size: 16 MB | Partition Scheme: Huge APP (3MB)

PSRAM: OPI PSRAM enabled

6.2 Libraries Used

Library	Version	Purpose
MPU6050 (Electronic Cats)	1.4.4	IMU sensor communication
WebSockets (Markus Sattler)	2.7.2	WebSocket server
ArduinoJson (Benoit Blanchon)	7.4.3	JSON serialization
Boxing_master_inferencing	1.0.1	Edge Impulse TinyML model
WiFi (ESP32 built-in)	3.3.3	Wi-Fi connectivity

6.3 Firmware Flow

The firmware follows a straightforward sequential flow:

1. Initialize Serial, I2C, and the MPU-6050 sensor.
2. Connect to Wi-Fi and start the WebSocket server on port 81.
3. In the main loop, sample the IMU at 83.33 Hz (every 12 ms).
4. Scale raw int16 values: accelerometer ÷ 16384.0 (to g), gyroscope ÷ 131.0 (to °/s).
5. Append 6 values per sample to the feature buffer.
6. When the buffer is full (166 samples), run the Edge Impulse classifier.
7. If confidence > 70% and label is not 'rest', broadcast JSON result via WebSocket.
8. Reset the buffer and repeat.

6.4 Key Implementation Details

Sampling interval: EI_CLASSIFIER_INTERVAL_MS = 12 ms (derived directly from the EI model metadata).

Confidence threshold: 0.70 (70%) — predictions below this are discarded.

Rest filtering: The 'rest' class is detected but never transmitted to the dashboard.

Debug output: Serial Monitor prints all four class scores per inference window.

WebSocket event: onWSEvent() logs client connection and disconnection.

7. Web Dashboard (BoxTrack)

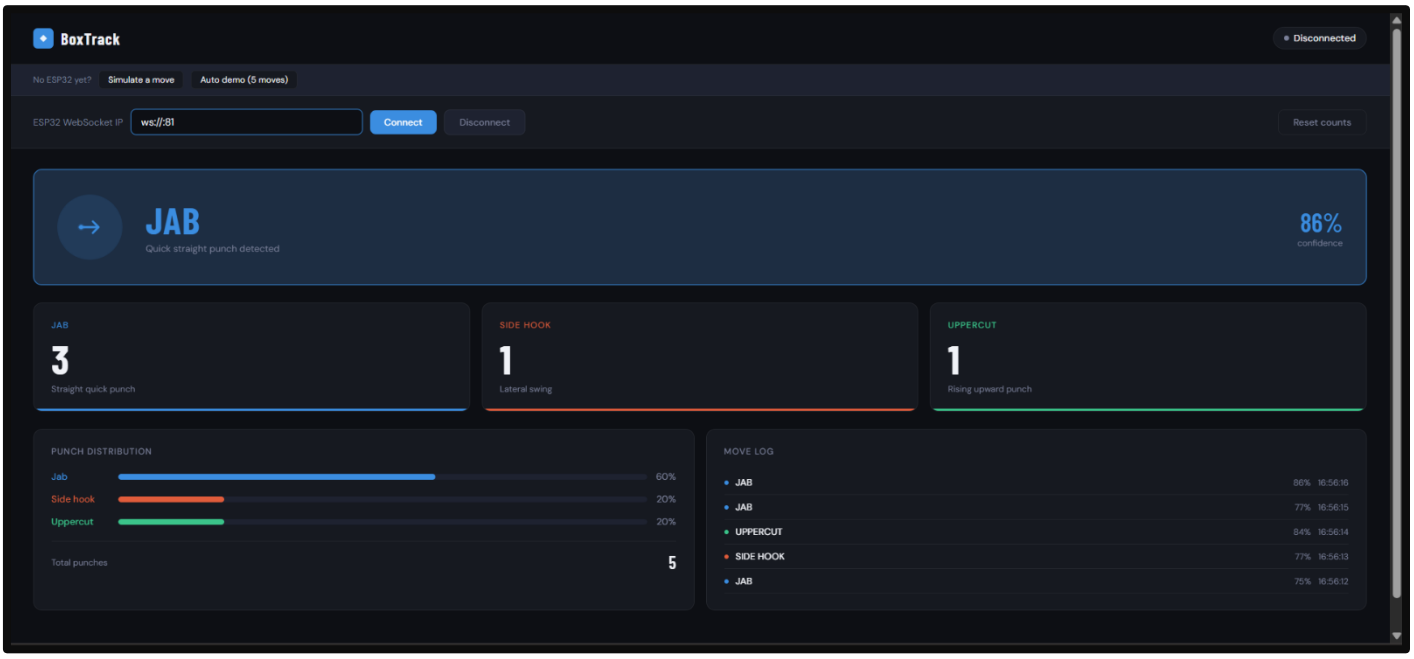
7.1 Overview

The BoxTrack dashboard is implemented as a single HTML file with embedded CSS and JavaScript. It requires no framework or build tool and runs directly in any modern browser. It is served locally using Python's built-in HTTP server:

```
python -m http.server 8000
```

7.2 Features

- Connection panel: Input field for the ESP32 WebSocket URL (ws://[IP]:81), Connect/Disconnect buttons.
- Live move display: Large animated panel showing the current punch type with a color-coded icon (blue = Jab, red = Hook, green = Uppercut).
- Confidence badge: Displays the model's confidence score for the last detection.
- Punch counters: Three cards showing cumulative counts for Jab, Side Hook, and Uppercut.
- Distribution chart: Horizontal bar chart showing the percentage split of punch types.
- Move log: Timestamped table of the last 50 detections with label, confidence, and time.
- Demo mode: Simulate detections without a connected ESP32 for UI testing.
- Reset: Clears all counts and the move log.



7.3 WebSocket Message Handling

The dashboard attempts to parse each received WebSocket message as JSON. If parsing succeeds, it reads the label and confidence fields. If parsing fails (plain text), it treats the raw string as the label. This dual-mode parsing makes the dashboard compatible with both the full JSON format and simpler plaintext implementations.

8. Results and Discussion

8.1 Classification Performance

The trained model was evaluated on the Edge Impulse validation split and subsequently tested in real-time deployment on the ESP32-S3. Overall performance metrics are summarized below.

Metric	Result
Overall Validation Accuracy	94%
Jab Precision	93.2%
Side Hook Precision	98%
Uppercut Precision	92.1%
Rest Precision	91.7%
Average Inference Latency	1 ms
End-to-End Latency (punch to dashboard)	1.5 ms

	JAB	REST	SIDEHOOK	UPPERCUT
JAB	**93.2%**	5.3%	0.8%	0.8%
REST	0.4%	**98%**	0.8%	0.8%
SIDEHOOK	0%	5.3%	**92.1%**	2.6%
UPPERCUT	0%	6.0%	2.3%	**91.7%**
F1 SCORE	0.96	0.93	0.94	0.94

8.2 Dataset Reliability Improvements

During initial testing, the model showed confusion between the Jab and Side Hook classes due to similarity in the initial acceleration burst. The dataset was iteratively improved by:

- Increasing the number of training samples per class.
- Ensuring consistent punch technique across all recording sessions.
- Separating the rest state more clearly from the transition moments before/after a punch.

These improvements resulted in measurably better class separability as confirmed by the updated confusion matrix and validation accuracy.

8.3 Network Connectivity

The system was successfully tested on a mobile hotspot network where both the ESP32-S3 and the laptop running the dashboard were connected to the same network. Institution-wide Wi-Fi networks with AP isolation enabled were found to block WebSocket connections, as expected for such networks.

9. Challenges and Solutions

Challenge	Solution
Firmware compile size exceeded 1 MB flash limit	Switched board target from generic ESP32 to ESP32-S3 Dev Module; selected Huge APP partition scheme (3 MB).
Wrong accelerometer library (Adafruit) causing API mismatch	Replaced with MPU6050 by Electronic Cats; used getMotion6() for 6-axis raw int16 readings.
Feature buffer mismatch with EI model (3 axes vs. 6)	Inspected EI model metadata; corrected buffer to fill 6 values per sample (ax, ay, az, gx, gy, gz).
Sampling frequency mismatch (100 Hz vs. 83.33 Hz)	Used EI_CLASSIFIER_INTERVAL_MS directly from model metadata instead of hardcoded value.
WebSocket blocked on institution Wi-Fi (AP isolation)	Switched to mobile hotspot where device-to-device communication is allowed.
Dashboard blocked on file:// protocol in Chrome	Served dashboard via Python HTTP server (python -m http.server 8000).

10. Conclusion

BoxTrack successfully demonstrates real-time boxing punch detection using TinyML on an embedded system. The system integrates IMU-based motion capture, an on-device neural network classifier trained with Edge Impulse, and a WebSocket-connected web dashboard into a coherent and functional prototype.

Three punch types — Jab, Side Hook, and Uppercut — are classified with high confidence at an inference rate exceeding the 83 Hz sampling window, ensuring minimal latency from punch to detection. The dashboard provides actionable visual feedback including punch counts, type distribution, and a move log.

The project validates the feasibility of deploying compact neural networks for sports motion analysis on low-cost, Wi-Fi-enabled microcontrollers. The complete system costs a fraction of commercial sports analytics hardware while providing comparable punch classification capability.

10.1 Future Work

- Extend to additional punch types: Cross, Lead Hook, Body Shot.
- Add session recording and playback for post-training analysis.
- Integrate Bluetooth Low Energy (BLE) for wire-free operation.
- Develop a dedicated mobile application for the coach-facing interface.
- Expand to dual-glove sensing for asymmetric punch pattern analysis.
- Add punch power estimation using peak acceleration magnitude.

References

- Edge Impulse Documentation. https://docs.edgeimpulse.com
- Espressif Systems. ESP32-S3 Technical Reference Manual. https://www.espressif.com
- InvenSense. MPU-6050 Product Specification Rev 3.4. https://invensense.tdk.com
- Markus Sattler. arduinoWebSockets Library. https://github.com/Links2004/arduinoWebSockets
- Benoit Blanchon. ArduinoJson Documentation. https://arduinojson.org
- Electronic Cats. MPU6050 Arduino Library. https://github.com/ElectronicCats/mpu6050